

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

KHOA CÔNG NGHỆ THÔNG TIN



BÁO CÁO TUẦN TÁM
MÔN THỰC TẬP CƠ SỞ

Giảng viên hướng dẫn : Kim Ngọc Bách

Sinh viên thực hiện : Nguyễn Trung Nghĩa

Mã Sinh Viên : B23DCCN602

Hà Nội - 2026

1. Mục tiêu hướng đến

Sau khi đã hoàn thiện hệ thống Fullstack cơ bản trong tuần thứ bảy với giao diện người dùng và các chức năng chính, tuần thứ tám của dự án tập trung vào việc tối ưu hóa hệ thống theo hướng realtime, nâng cao hiệu năng xử lý và cải thiện trải nghiệm người dùng trong môi trường vận hành thực tế.

Các mục tiêu chính trong tuần này bao gồm:

- Tối ưu hóa luồng livestream theo hướng tiết kiệm tài nguyên và ổn định lâu dài
- Xây dựng cơ chế quản lý camera tập trung cho nhiều người dùng
- Đồng bộ dữ liệu realtime giữa Backend và Frontend thông qua WebSocket
- Cải thiện hệ thống thống kê và báo cáo với dữ liệu chính xác hơn
- Nâng cấp cơ chế quản lý trạng thái phiên làm việc trên Frontend
- Tối ưu trải nghiệm người dùng và loại bỏ các thao tác không cần thiết

Việc hoàn thành các mục tiêu này giúp hệ thống chuyển từ mức “hoạt động được” sang “vận hành ổn định, tối ưu và gần với thực tế triển khai”.

2. Tối ưu hóa hệ thống livestream và xử lý realtime

Để đạt được hiệu suất tối đa, em đã thực hiện những cải tiến quan trọng về mặt kiến trúc xử lý đa nhiệm và truyền tải dữ liệu.

2.1. Kiến trúc Global Camera và Luồng kép (Dual-stream)

Em đã tái cấu trúc hoàn toàn cách Backend quản lý Camera thông qua lớp GlobalCamera. Thay vì mỗi request mở một luồng camera riêng gây lãng phí tài nguyên, hệ thống hiện tại hoạt động theo cơ chế:

- Luồng 1 (Capture Thread): Sử dụng threading để đọc khung hình thô từ phần cứng với tốc độ cao nhất có thể.
- Luồng 2 (Inference Task): Một tác vụ asyncio chạy song song để lấy khung hình mới nhất, thực hiện nhận diện AI và broadcast kết quả qua WebSocket.
- Quản lý Viewers: Hệ thống tự động đếm số lượng người xem. Camera chỉ thực sự hoạt động khi có ít nhất một kết nối và sẽ tự động giải phóng tài nguyên khi người dùng cuối cùng thoát ra.

2.2. Tối ưu vòng lặp inference của AI

Một vấn đề lớn trong hệ thống realtime là việc mô hình AI xử lý quá nhiều frame liên tục, gây quá tải tài nguyên. Vì vậy em đã thiết lập cơ chế chỉ cho AI chạy mỗi 2 frame và giữ delay nhỏ mỗi lần detect như đã làm trong các tuần trước:

- Đếm số frame đã nhận được và chia module cho 2. Nếu chia hết cho 2 thì chạy AI để detect frame đó còn không thì lấy dữ liệu của frame trước đó.
- Do 2 frame liên tiếp nên sự chênh lệch của đối tượng rất nhỏ, do vậy bounding box vẫn đúng mà lại có thể giảm tải cho server, và video hiện lên vẫn rất mượt.

2.3. Triển khai WebSockets và Broadcaster

Đây là bước ngoặt về mặt công nghệ trong tuần này:

- Endpoint /ws: Xây dựng hệ thống WebSocketManager để quản lý danh sách các kết nối WebSocket đang hoạt động.
- Cơ chế đẩy (Push): Ngay khi một vụ vi phạm được lưu vào database, Backend sẽ phát tín hiệu (broadcast) kèm theo dữ liệu vi phạm đến toàn bộ các Dashboard đang mở. Người quản lý sẽ nhận được cảnh báo ngay lập tức mà không cần tải lại trang.
- Xác thực WebSocket: Do giao thức WS không hỗ trợ Header tùy chỉnh khi khởi tạo, em đã nâng cấp get_current_user để chấp nhận Token qua Query Parameter, đảm bảo luồng dữ liệu thời gian thực vẫn được bảo mật 24/7.

2.4. Ép hiệu năng phần cứng (GPU & FP16)

Để xử lý mượt mà video HD 720p, em đã tối ưu tham số nạp mô hình:

- CUDA Acceleration: Sử dụng .to('cuda') để ép chuyển toàn bộ tính toán AI sang GPU.
- Half-precision (FP16): Kích hoạt tham số half=True trong quá trình predict, giúp giảm một nửa dung lượng bộ nhớ video và tăng tốc độ xử lý mà không làm giảm đáng kể độ chính xác.

2.5. Chuẩn hóa hệ thống ghi nhật ký (Logging)

Nhằm nâng cấp hệ thống lên tiêu chuẩn vận hành công nghiệp và hỗ trợ quá trình truy vết lỗi (Debugging) hiệu quả hơn, em đã thực hiện thay thế toàn bộ các lệnh print thủ công bằng thư viện logging chuyên nghiệp:

- **Quản lý theo cấp độ (Log Levels):** Thay vì in dữ liệu tràn lan ra console, các sự kiện hiện được phân loại rõ ràng theo mức độ nghiêm trọng như INFO (thông tin vận hành), WARNING (cảnh báo) và ERROR (lỗi hệ thống). Điều này đặc biệt quan trọng trong các module nhạy cảm như xác thực và gửi Email.
- **Cấu trúc hóa thông tin:** Mỗi dòng log hiện tại đều đi kèm với định dạng chuẩn bao gồm: Thời gian xảy ra - Tên Module - Cấp độ lỗi - Nội dung chi tiết. Điều này giúp việc kiểm soát luồng dữ liệu của Backend trở nên minh bạch và dễ dàng quản lý thông qua các công cụ đọc log tập trung.
- **Hiệu quả vận hành:** Việc sử dụng logger giúp giảm thiểu việc ghi dữ liệu thừa thãi vào luồng xuất tiêu chuẩn, từ đó tối ưu hóa hiệu suất cho máy chủ khi chạy ở chế độ Production.

3. Nâng cấp Trải nghiệm Frontend (UX/UI)

Giao diện Dashboard được tinh chỉnh để phản ánh trạng thái “Neural Engine” một cách chuyên nghiệp và trực quan hơn.

3.1. Hệ thống HUD và Hiệu ứng thị giác

Trong tuần này, em tập trung cải thiện trải nghiệm người dùng bằng cách bổ sung các thành phần HUD và hiệu ứng hiển thị trực quan trên giao diện giám sát:

- **Scanline Effect:** Thêm lớp phủ đường quét radar trên khung hình Livestream, tạo phong cách Tactical Oversight đồng nhất cho dự án.
- **Trạng thái Engine:** Header hiện tại hiển thị trạng thái kết nối thời gian thực của “Neural Engine” và “Sync” thông qua màu sắc và hiệu ứng nhấp nháy (Online/Connecting/Offline).
- **Real-time Notifications:** Chuông thông báo hiển thị số lượng vi phạm mới chưa đọc. Khi click vào, người dùng sẽ được chuyển hướng đến trang lịch sử và dòng vi phạm đó sẽ được highlight với hiệu ứng Pulse (nhấp xanh) để dễ dàng nhận diện.

3.2. Cải tiến trang Phân tích

Nhằm mang lại cái nhìn sâu sắc và quy trình làm việc mượt mà hơn, trang Phân tích đã được cập nhật các tính năng và cải tiến giao diện quan trọng sau:

- **Accuracy Tracking:** Dataset “Accuracy” đã được bổ sung vào biểu đồ xu hướng. Dữ liệu này được tính toán dựa trên độ tin cậy trung bình của các lần nhận diện, giúp người quản lý đánh giá được điều kiện ánh sáng và góc quay của camera có đang đạt yêu cầu hay không.
- **Skeleton Loading:** Thay thế các vòng quay tải dữ liệu (Spinner) bằng hệ thống Skeleton (khung xương dữ liệu) mượt mà, giúp giảm cảm giác chờ đợi khi hệ thống đang xử lý các truy vấn lớn.
- **Sửa lỗi logic “Yesterday”:** Chỉ lấy dữ liệu của ngày hôm qua thay vì lấy cả 2 ngày hôm qua và hôm nay

3.3. Tối ưu cơ chế Reset Session

Em đã thực hiện thay thế phương thức reset cũ bằng cơ chế In-app Reset logic để khắc phục tình trạng mất trạng thái và gián đoạn trải nghiệm người dùng:

- **Logic xử lý:** Thay vì tải lại trang, hệ thống thực hiện cập nhật lại mốc `session_start` trong `localStorage`, đồng thời reset trực tiếp các biến đếm và danh sách ID vi phạm thông qua React State và WebSocket service.
- **Kết quả:** Phiên làm việc được làm mới tức thì, duy trì kết nối ổn định với Neural Engine mà không gây hiện tượng “trắng trang” hay trễ mạng.

4. Kết quả đạt được trong tuần 8

Sau khi hoàn thành các công việc trong tuần thứ tám, các kết quả chính đạt được bao gồm:

- Tối ưu hoàn toàn hệ thống livestream theo hướng tiết kiệm tài nguyên
- Xây dựng thành công cơ chế Global Camera cho nhiều người dùng
- Giảm tải đáng kể CPU/GPU trong quá trình inference
- Nâng cấp hệ thống báo cáo với chỉ số Accuracy
- Hoàn thiện hệ thống WebSocket realtime
- Cải thiện đáng kể trải nghiệm người dùng (UX)
- Loại bỏ hoàn toàn các thao tác reload không cần thiết
- Đồng bộ dữ liệu realtime giữa Backend và Frontend

5. Khó khăn gặp phải và cách khắc phục

Trong tuần thứ tám của dự án, quá trình tối ưu hóa hệ thống realtime và nâng cấp kiến trúc xử lý video đã phát sinh nhiều thách thức kỹ thuật liên quan đến việc quản lý tài nguyên camera, đồng bộ dữ liệu giữa nhiều client và đảm bảo hiệu năng xử lý AI trong môi trường hoạt động liên tục. Các vấn đề này chủ yếu xuất phát từ yêu cầu phải duy trì luồng livestream ổn định cho nhiều người dùng đồng thời, đồng thời đảm bảo dữ liệu được cập nhật theo thời gian thực mà không gây quá tải hệ thống.

6.1. Duy trì trạng thái dữ liệu thống kê (Session Persistence)

Khó khăn: Ban đầu, các chỉ số như số vụ vi phạm và độ chính xác được quản lý tại State của từng Component. Điều này dẫn đến việc dữ liệu bị reset về 0 mỗi khi người dùng chuyển đổi giữa các trang (như từ trang Giám sát sang Lịch sử hoặc Phân tích).

Giải pháp: Em đã di chuyển toàn bộ logic quản lý dữ liệu vào một Singleton Service (websocket.js). Bằng cách này, dữ liệu được lưu trữ tập trung và bền vững trong suốt vòng đời ứng dụng, không còn phụ thuộc vào trạng thái hiển thị của giao diện.

6.2. Xung đột tài nguyên khi truy cập đa người dùng

Khó khăn: Xảy ra lỗi “Camera Device Busy” hoặc treo luồng video khi có nhiều phiên làm việc cùng truy cập trang giám sát. Nguyên nhân là do hệ thống cố gắng khởi tạo nhiều tiến trình mở Camera phân cứng cùng lúc.

Giải pháp: Triển khai kiến trúc Global Camera Singleton. Máy chủ hiện tại chỉ duy trì một thực thể Camera duy nhất, mọi kết nối từ các máy khách khác nhau sẽ cùng chia sẻ chung một luồng truyền tải, giúp tối ưu hóa tài nguyên phần cứng tuyệt đối.

6.3. Mất đồng bộ giữa tốc độ hiển thị và tốc độ xử lý AI

Khó khăn: Tốc độ nạp khung hình của Camera nhanh hơn tốc độ tính toán của mô hình AI, gây ra hiện tượng “nghẽn cổ chai” khiến video bị khựng lag để chờ kết quả Inference.

Giải pháp: Áp dụng Kiến trúc luồng kép (Dual-Stream): luồng Capture chuyên trách việc đọc ảnh và luồng Inference chuyên trách chạy AI. Kết hợp với kỹ thuật Frame Skipping, giúp video livestream duy trì độ mượt mà ổn định.

7. Kế hoạch thực hiện trong tuần tiếp theo

Sau khi đã tối ưu hóa hệ thống realtime và hoàn thiện cơ chế đồng bộ dữ liệu trong tuần thứ tám, dự án đã đạt đến mức độ ổn định cao về mặt kỹ thuật. Trong tuần tiếp theo, các công việc sẽ tập trung vào việc hoàn thiện trải nghiệm người dùng, tối ưu hiệu năng tổng thể và nâng cao chất lượng mô hình AI.

7.1. Cải thiện hiệu năng hệ thống

Mặc dù hệ thống hiện tại đã có thể vận hành ổn định, tuy nhiên trong một số trường hợp, thời gian phản hồi vẫn còn độ trễ nhất định.

Trong tuần tới, em dự kiến cải thiện hiệu năng thông qua việc caching dữ liệu theo như mục tiêu của tuần trước chưa hoàn thành

7.2. Tiếp tục tăng cường độ chính xác cho mô hình

Trong quá trình kiểm thử thực tế thông qua webcam trên giao diện web, em nhận thấy độ tin cậy của mô hình còn có sự dao động đáng kể giữa các lần phát hiện.

Nguyên nhân có thể bao gồm:

- Điều kiện ánh sáng không ổn định
- Chất lượng hình ảnh đầu vào thấp (blur, noise)
- Tập dữ liệu huấn luyện chưa đủ đa dạng

Trong tuần tiếp theo, em sẽ thực hiện các hướng cải thiện như:

- Bổ sung và đa dạng hóa dữ liệu huấn luyện
- Tuning lại các tham số của mô hình
- Xem xét áp dụng các kỹ thuật tiền xử lý ảnh

Mục tiêu là nâng cao độ ổn định và độ chính xác của hệ thống nhận diện trong điều kiện thực tế.